

C++ Space Telemetry Ground Station

Dossier technique pour recruteur informatique

Version française

Projet portfolio C++20 - Linux - télémétrie - réseau - multithreading - CMake - tests - CI/CD

Objectif du document

Présenter le projet à un recruteur ou à un ingénieur logiciel afin d'expliquer les intentions de réalisation, l'architecture, les choix de développement, la qualité logicielle et les compétences démontrées par le code.

Auteur

Guillaume Lemonnier

Date

Juillet 2026

Table des matières

1 Résumé exécutif	2
2 Volonté de réalisation	2
2.1 Démontrer du C++ moderne concret	2
2.2 Rapprocher le projet d'un contexte industriel	2
2.3 Produire un projet explicable en entretien	2
3 Périmètre fonctionnel livré	3
4 Architecture générale	3
4.1 Organisation du code	3
5 Choix de développement	4
5.1 Choix de C++20	4
5.2 Choix de CMake	4
5.3 Choix du modèle d'erreur	5
5.4 Choix d'un protocole binaire strict	5
5.5 Choix UDP et TCP	5
6 Pipeline multithreadé	5
7 Mode dégradé	6
7.1 Intention	6
7.2 Mécanisme	6
8 Replay, export et observabilité	6
8.1 Format de replay STGF	6
8.2 Exports CSV et JSON	6
8.3 Logs	7
9 Tests, qualité et CI/CD	7
9.1 Stratégie de test	7
9.2 GitHub Actions	7
10 Benchmark et performance	7
11 Compétences démontrées	8
12 Limites assumées	8
13 Pistes d'évolution	8
14 Message à tenir en entretien	9
15 Commandes de démonstration	9
15.1 Build et tests	9
15.2 Replay vers CSV et JSON	9
15.3 Simulation UDP avec pertes et corruption	9
15.4 Benchmark	9
16 Conclusion	9

1 Résumé exécutif

C++ Space Telemetry Ground Station est un projet C++20 sous Linux qui simule une station sol capable de recevoir, décoder, valider, rejouer et exporter des trames de télémétrie de type satellite ou équipement embarqué. Le projet a été pensé comme une preuve de compétence pour des postes orientés logiciel industriel, systèmes, télémétrie, réseau, C++ moderne et environnements contraints.

Le projet couvre une chaîne complète : génération de trames, réception UDP/TCP, re-construction de flux TCP, parsing binaire strict, validation CRC-32, pipeline multithreadé, fichiers de replay, export CSV/JSON, journalisation, tests automatisés, CI GitHub Actions, benchmark reproductible et mode opérationnel dégradé.

Positionnement recruteur

Ce projet ne cherche pas à être un produit spatial certifié. Il sert à démontrer une capacité à construire un mini-système C++ structuré, testable, robuste et explicable, avec des choix proches de problématiques rencontrées dans des environnements industriels, défense, spatial ou embarqués.

2 Volonté de réalisation

Le projet répond à trois intentions principales.

2.1 Démontrer du C++ moderne concret

L'objectif n'était pas de produire un simple exercice de syntaxe C++, mais un projet où le langage est utilisé pour structurer un vrai pipeline applicatif : modèles de domaine, encodage binaire, gestion d'erreurs, I/O Linux, files thread-safe, tests et outils CLI.

Les éléments travaillés sont notamment :

- C++20, RAII, types forts et séparation en modules ;
- usage de `std::variant` pour distinguer une trame valide d'une erreur de parsing ;
- files bloquantes thread-safe pour découpler réception, décodage et écriture ;
- compilation stricte avec CMake, warnings et tests automatisés.

2.2 Rapprocher le projet d'un contexte industriel

Le domaine de la télémétrie est volontairement choisi car il oblige à traiter des sujets proches du logiciel système : protocole binaire, intégrité, perte réseau, corruption de données, replay, monitoring d'état et performances.

Le projet inclut donc :

- un protocole binaire strict ;
- une validation CRC avant confiance dans les données ;
- une simulation de pertes et de corruption ;
- un mode dégradé lorsque le flux devient trop instable ;
- un benchmark pour mesurer le comportement du pipeline.

2.3 Produire un projet explicable en entretien

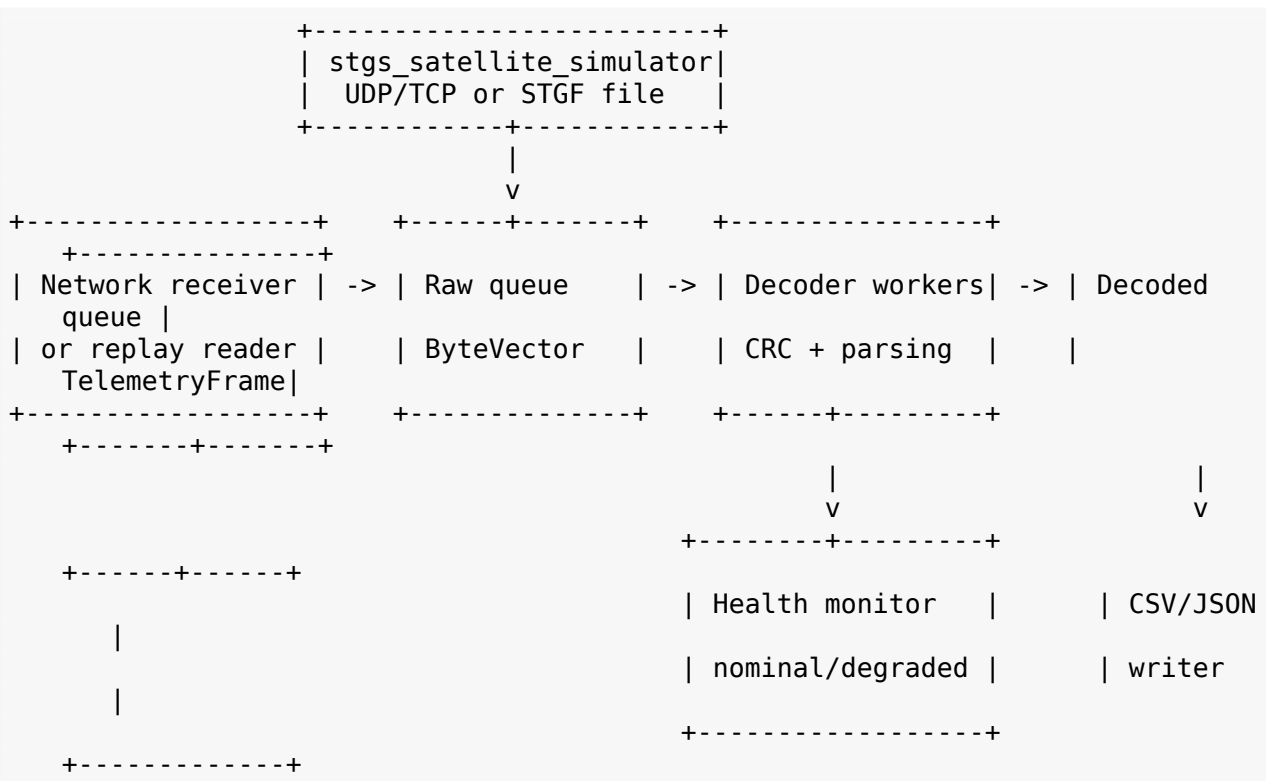
Le code a été organisé pour être facilement présenté : arborescence claire, documentation technique, README anglais, commandes reproductibles et séparation entre code métier, infrastructure réseau, replay, tests et benchmark.

3 Périmètre fonctionnel livré

Version	Objectif	Fonctionnalités implémentées
V1	Pipeline de base	Simulateur de trames, réception UDP, parser C++, validation CRC-32, logs, tests unitaires, README anglais.
V2	Industrialisation	Pipeline multithreadé, fichiers de replay STGF, export CSV et JSON, benchmark reproductible, CTest, GitHub Actions.
V3	Robustesse opérationnelle	Protocole binaire plus strict, simulation de pertes réseau et corruptions, mode dégradé, documentation d'architecture, rapport de performance.

4 Architecture générale

L'architecture est volontairement découpée en composants simples. La station sol peut recevoir des trames depuis le réseau ou depuis un fichier de replay. Ces trames brutes sont placées dans une file, décodées par un ou plusieurs workers, contrôlées par un moniteur de santé, puis exportées.



4.1 Organisation du code

Fichier ou dossier	Responsabilité
--------------------	----------------

apps/ground_station.cpp	Application principale : CLI, réception UDP/TCP, replay, workers, exports, mode dégradé.
apps/satellite_simulator.cpp	Générateur de télémétrie : réseau, fichier STGF, pertes, corruption, débit simulé.
include/stgs/TelemetryFrame.hpp	Modèle de domaine représentant une trame décodée.
include/stgs/FrameCodec.hpp	Encodage, décodage, validation et
src/FrameCodec.cpp	extraction de trames dans un flux TCP.
include/stgs/NetworkServer.hpp	Réception UDP/TCP sous Linux via
src/NetworkServer.cpp	sockets POSIX et poll().
include/stgs/BlockingQueue.hpp	File producteur/consommateur
	fermable pour le pipeline multithreadé.
include/stgs/StationHealth.hpp	Surveillance de santé opérationnelle et
src/StationHealth.cpp	transitions nominal/dégradé.
include/stgs/Replay.hpp	Lecture/écriture du format de replay
src/Replay.cpp	STGF.
benchmarks/benchmark_decode.cpp	Benchmark reproductible du pipeline de
	décodage.
tests/test_main.cpp	Tests unitaires et tests de
	comportement.
.github/workflows/ci.yml	Compilation, tests, smoke replay,
	export JSON et benchmark smoke.

5 Choix de développement

5.1 Choix de C++20

C++20 est utilisé pour rester proche de postes système/industriel tout en bénéficiant d'un langage plus expressif et plus sûr qu'un style C++ ancien. Le projet privilégie une approche explicite : types métier, fonctions isolées, composants testables, peu de dépendances externes.

Pourquoi peu de dépendances ?

Le projet est volontairement léger. L'objectif est de montrer la maîtrise des fondamentaux C++/Linux plutôt que de masquer la complexité derrière une pile de frameworks. Cela rend aussi le build plus simple en CI et facilite l'audit du code.

5.2 Choix de CMake

CMake structure le projet en cibles séparées. Les principales cibles sont :

- stgs_core pour les composants de base ;
- stgs_network pour l'infrastructure réseau ;
- stgs_ground_station pour l'application de réception ;
- stgs_satellite_simulator pour le générateur ;
- stgs_tests pour CTest ;
- stgs_benchmark_decode pour les mesures de performance.

Cette séparation permet de compiler le coeur indépendamment des applications et de réutiliser les mêmes composants dans les tests.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

```
ctest --test-dir build --output-on-failure
```

5.3 Choix du modèle d'erreur

Le parser retourne un `std::variant<TelemetryFrame, FrameError>`. Une trame invalide est considérée comme un cas de données attendu, pas comme une exception applicative.

Ce choix est pertinent pour un flux de télémétrie : dans un réseau bruité, recevoir des données invalides n'est pas un crash, mais un événement à compter, journaliser et utiliser pour décider d'un état dégradé.

```
using FrameParseResult = std::variant<TelemetryFrame, FrameError>;  
  
FrameParseResult decodeFrame(std::span<const std::uint8_t> bytes);
```

5.4 Choix d'un protocole binaire strict

Le protocole contient un mot magique, une version, un identifiant satellite, un timestamp, des mesures, une longueur de payload et un CRC-32. Tous les entiers sont en big-endian pour être cohérents avec l'ordre réseau.

```
MAGIC | VERSION | SATELLITE_ID | TIMESTAMP_MS | TEMPERATURE_C |  
BATTERY_PERCENT | STATUS | PAYLOAD_LEN | PAYLOAD | CRC32
```

Les règles de validation sont strictes : magic attendu, version supportée, longueur cohérente, taille maximale de payload, CRC correct, batterie entre 0 et 100, statut connu.

Pourquoi un magic word ?

En TCP, le transport est un flux d'octets et ne conserve pas les frontières applicatives. Le mot magique STGS permet de resynchroniser le flux si des octets parasites ou des trames corrompues apparaissent.

5.5 Choix UDP et TCP

Le projet implémente les deux approches :

- en UDP, chaque datagramme est traité comme une trame candidate complète ;
- en TCP, un extracteur reconstruit les trames à partir d'un flux d'octets.

Cela permet d'illustrer deux modèles réseau différents : la simplicité et la perte possible côté UDP, la fiabilité de transport mais la nécessité de framing côté TCP.

6 Pipeline multithreadé

Le pipeline sépare les responsabilités : réception, décodage, surveillance et écriture. Le nombre de workers de décodage est configurable via `--decoder-threads`. Cette conception montre une compréhension des problématiques de concurrence sans introduire de complexité inutile.

```
Receiver or replay reader  
-> BlockingQueue<ByteVector>  
    -> decoder worker 1  
    -> decoder worker 2  
    -> ...  
    -> BlockingQueue<TelemetryFrame>  
        -> CSV/JSON writer
```

Le composant `BlockingQueue` est fermable, ce qui simplifie l'arrêt propre des threads. Lorsqu'il n'y a plus de données, les workers peuvent sortir proprement sans attente active.

7 Mode dégradé

7.1 Intention

Le mode dégradé est une fonctionnalité volontairement ajoutée pour dépasser le simple parsing de trames. Il introduit une notion d'état opérationnel de la station : même si le programme continue de fonctionner, le flux de télémétrie peut devenir suffisamment mauvais pour devoir signaler une dégradation.

7.2 Mécanisme

`StationHealthMonitor` maintient une fenêtre glissante d'échantillons récents. Chaque échantillon correspond soit à une trame rejetée, soit à une trame décodée. Une trame décodée est marquée critique si son statut vaut `CRITICAL` ou `SAFE_MODE`.

La station passe de `NOMINAL` à `DEGRADED` si :

- le taux de rejet dépasse le seuil configuré ;
- ou le nombre de trames critiques atteint le seuil configuré.

Elle revient à `NOMINAL` lorsque le taux de rejet repasse sous le seuil de récupération et que la fenêtre ne contient plus de volume critique problématique.

```
--degraded-window 100
--degraded-min-samples 20
--degraded-rejection-rate 0.10
--degraded-recovery-rate 0.03
--degraded-critical-count 3
```

Valeur pour un recruteur

Cette partie montre une capacité à penser au comportement opérationnel d'un logiciel, pas seulement à son résultat nominal. Le code surveille les erreurs, prend une décision d'état et journalise les transitions.

8 Replay, export et observabilité

8.1 Format de replay STGF

Le format STGF permet d'enregistrer des trames binaires puis de les rejouer dans le même pipeline que le réseau. C'est un choix important pour le test, la reproductibilité et l'analyse d'incident.

```
'S' 'T' 'G' 'F' 0 0 0 1
FRAME_LENGTH:uint32_be | FRAME_BYTES
```

8.2 Exports CSV et JSON

Le CSV permet une inspection simple avec des outils bureautiques. Le JSON facilite l'intégration avec des outils d'analyse, dashboards ou tests automatisés. Le format peut être inféré depuis l'extension ou forcé via `--output-format`.

```
./build/stgs_ground_station --replay frames.stgf --output replay.csv  
./build/stgs_ground_station --replay frames.stgf --output replay.json  
./build/stgs_ground_station --replay frames.stgf --output telemetry.out --  
output-format json
```

8.3 Logs

Le logger est thread-safe et peut écrire en console ou dans un fichier. Les logs servent à comprendre le comportement de la station : démarrage, erreurs de trames, transitions de santé, résumé de fin.

9 Tests, qualité et CI/CD

9.1 Stratégie de test

Les tests vérifient les comportements critiques : CRC connu, round trip encode/décode, rejet des erreurs de magic, CRC, longueur, extraction TCP avec bruit, replay, queue bloquante et transitions du mode dégradé.

```
ctest --test-dir build --output-on-failure
```

9.2 GitHub Actions

Le workflow CI compile en Release, exécute CTest, génère un replay STGF, rejoue vers CSV, rejoue vers JSON, valide le JSON avec Python et lance un benchmark smoke. Cela montre que le projet est pensé pour être vérifié automatiquement, pas seulement exécuté localement.

Choix qualité

Pour un recruteur, la présence de CTest, d'un workflow CI, d'un benchmark et de tests d'erreurs est aussi importante que la fonctionnalité nominale. Cela démontre une culture de maintenabilité et d'industrialisation.

10 Benchmark et performance

Le projet inclut stgs_benchmark_decode, un outil qui mesure le débit du pipeline de décodage sans réseau ni disque. Il pré-génère des trames valides, les pousse dans la file, les décode avec un nombre configurable de workers et mesure les frames par seconde.

```
./build/stgs_benchmark_decode --frames 200000 --payload-size 64 --decoder-  
threads 4
```

Le rapport de performance documente la méthode et des mesures de référence. Les résultats montrent notamment que le multithreading n'améliore pas automatiquement les performances sur de petites charges : la synchronisation de file peut coûter plus cher que le travail de parsing. Cette interprétation est utile en entretien car elle montre que les choix de concurrence doivent être mesurés.

11 Compétences démontrées

Compétence	Preuve dans le projet
C++ moderne	C++20, types de domaine, <code>std::variant</code> , RAII, séparation en bibliothèques.
Linux	Sockets POSIX, <code>poll()</code> , CLI, fichiers, exécution et build Linux.
Réseau	Réception UDP/TCP, framing TCP, simulation de pertes et corruption.
Protocole binaire	Magic word, version, longueurs, big-endian, CRC, payload variable.
Multithreading	Queues thread-safe, workers de décodage, writer séparé, arrêt propre.
Performance	Benchmark reproductible, mesure frames/seconde, analyse d'impact payload/threads.
CMake	Cibles séparées, options de build tests/benchmarks, CTest.
Tests	Tests unitaires et tests d'erreurs, intégration CTest et CI.
CI/CD	GitHub Actions avec build, tests, replay, JSON validation et benchmark smoke.
Documentation	README anglais, documentation d'architecture et rapport de performance.
Robustesse	Rejet des trames invalides, monitoring de santé, mode dégradé, logs.

12 Limites assumées

Le projet est volontairement pédagogique et portfolio. Il ne prétend pas couvrir les exigences d'un système spatial certifié. Les principales limites sont :

- protocole inspiré télémétrie, mais non compatible CCSDS ;
- pas de vraie couche hardware ou driver ;
- pas de persistance longue durée ni rotation de fichiers ;
- pas de métriques Prometheus ou dashboard web ;
- pas de fuzzing ni tests de charge réseau complets ;
- pas de packaging système type Debian ou service systemd.

Ces limites sont utiles à mentionner en entretien car elles montrent une capacité à distinguer un projet portfolio d'un logiciel de production critique.

13 Pistes d'évolution

Les évolutions les plus pertinentes pour renforcer encore le projet seraient :

- ajouter un header inspiré CCSDS avec compteur de séquence ;
- calculer la perte UDP à partir des compteurs de trames ;
- ajouter un dashboard WebSocket ou Prometheus ;
- ajouter des tests de fuzzing sur le parser ;
- créer un mode service Linux avec systemd ;
- ajouter une simulation de périphérique embarqué ou capteur matériel ;
- ajouter des scénarios de charge réseau plus réalistes ;

- produire une documentation Doxygen du coeur C++.

14 Message à tenir en entretien

J'ai développé ce projet pour démontrer ma montée en compétence sur du C++ moderne orienté systèmes. L'objectif était de construire une mini station sol de télémétrie capable de recevoir des trames UDP/TCP ou des fichiers de replay, de les décoder via un protocole binaire strict, de valider leur intégrité par CRC, d'exporter les données, de mesurer les performances et de surveiller l'état opérationnel via un mode dégradé. Le projet montre surtout ma capacité à structurer un code C++ testable, documenté, compilé avec CMake, vérifié par CI et explicable techniquement.

15 Commandes de démonstration

15.1 Build et tests

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure
```

15.2 Replay vers CSV et JSON

```
./build/stgs_satellite_simulator --output-file frames.stgf --count 1000 --rate 0
./build/stgs_ground_station --replay frames.stgf --output replay.csv
./build/stgs_ground_station --replay frames.stgf --output replay.json
```

15.3 Simulation UDP avec pertes et corruption

```
./build/stgs_ground_station --udp --port 9000 --output telemetry.json --log ground.log
./build/stgs_satellite_simulator --udp --host 127.0.0.1 --port 9000 \
--count 5000 --rate 2000 --loss 0.02 --corrupt 0.01
```

15.4 Benchmark

```
./build/stgs_benchmark_decode --frames 200000 --payload-size 64 --decoder-threads 4
./benchmarks/run_benchmark.sh
```

16 Conclusion

C++ Space Telemetry Ground Station est un projet portfolio orienté recruteur qui démontre une base solide en C++ moderne, Linux, réseau, protocole binaire, multithreading, CMake, tests, CI/CD, robustesse et documentation technique. Sa valeur principale est de montrer une démarche complète : partir d'un objectif technique clair, concevoir

une architecture modulaire, implémenter les fonctionnalités, tester les erreurs, mesurer les performances et documenter les choix.

Pour un poste de développement logiciel industriel, spatial, défense, systèmes ou télémétrie, ce projet constitue une preuve concrète d'apprentissage structuré et de capacité à produire un code explicable, maintenable et vérifiable.